Toward Generative Design of Aircraft-Like Structures with Diffusion Models and CFD-Informed Evaluation

Darsh Gupta, National Public School, HSR Layout, Bengaluru
darshgu77@gmail.com

***Abstract*—This paper documents a reproducible latent generative-design pipeline for aircraft-like voxel geometries, with a CFD-informed scoring path and a validation framework that separates code-path evidence from stronger aerodynamic, structural, and conditioning claims. The repository combines a latent diffusion-style model, a voxel decoder (dense below 96^3, coordinate-based above), an interruption-safe monitored training runner with collapse-recovery safeguards, and an internal D3Q27 lattice-Boltzmann solver plus an external OpenFOAM export route. The present evidence rests on public VSP Airshow smoke runs, a deterministic reference evidence package, and a 642-test automated suite that pins the architectural and recovery contracts. Within this scope, the codebase supports a proof-of-concept claim and a reproducible reduced evidence bundle; validated mission-conditioned or manufacturing-conditioned aircraft generation, structural load-path analysis, and publication-grade aerodynamic or structural validation remain explicitly future work until the documented final-run gates pass.**

***Index Terms*—Generative Design, Diffusion Models, Computational Fluid Dynamics, Aerospace Engineering, Voxel Geometry.**

I. Introduction

GENERATIVE design is increasingly used to explore large engineering design spaces, while aerospace workflows continue to rely heavily on computational fluid dynamics (CFD), surrogate models, and optimization loops to evaluate aerodynamic tradeoffs [1], [2]. Aircraft design adds further constraints beyond raw shape generation, including manufacturability, structural load paths, and mission-specific performance targets. Those requirements make it easy to overstate what a proof-of-concept generative repository can presently demonstrate.

This paper therefore takes a deliberately narrower position. The repository implements a latent diffusion-style pipeline that maps latent samples to voxel geometries, scores them with an internal CFD-oriented path, and exports them for downstream validation. The present evidence comes from synthetic training data and a reduced final protocol, so the goal of this paper is not to claim a finished aircraft-design system. Instead, the goal is to document the implementation, report what the current code path reproduces, and make the boundary between demonstrated behavior and future work explicit.

The key contributions of this work are:

1) A reproducible proof-of-concept latent generative pipeline for producing aircraft-like voxel geometries and exporting them to mesh-based artifacts, with a dense-to-coordinate decoder split and a documented 22-slot structured-conditioning vector routed end to end.
2) An interruption-safe monitored training runner with run-state checkpointing, exact-resume fingerprinting, and collapse-recovery safeguards (warm-start gates, geometry-threshold preservation, and fail-closed numerical guards), together with a 642-test automated suite that pins these contracts.
3) An implementation path for CFD-informed evaluation that couples an internal lattice-Boltzmann-based scorer (D3Q27) with a black-box SPSA solver-in-loop training term and an external OpenFOAM benchmark route, together with reduced sanity experiments showing that the end-to-end path executes.
4) An issue-driven validation framework that distinguishes code-path validation from stronger claims such as aircraft-specific validity, aerodynamic optimization, structural viability, and conditioned generation from mission or manufacturing constraints, and a single documented final-run protocol whose gates the remaining claims are bound to.

The remainder of the paper first places the repository beside 3D generative modeling, topology optimization, and CFD-driven design practice, then describes the implementation that exists in the codebase. It then reports the reduced final-protocol evidence, records the validation requirements that still block stronger claims, and closes by summarizing the present scope and the minimum future work needed for a conditioned airplane generator.

II. Related Work

Diffusion models are now a standard point of reference for high-quality generative modeling [3], [4]. In 3D generation, recent systems such as Point-E and Shap-E show that diffusion-style methods can generate point-cloud or implicit 3D assets at practical sampling speeds [5], [6]. Those works are important context because they establish that diffusion models can represent 3D geometry, but they do not by themselves solve aircraft-specific design, manufacturability, or validation concerns.

An adjacent engineering-design tradition comes from topology optimization, where the goal is to optimize material layout or geometry under physics-based objectives and constraints. That literature is mature and has a well-developed

vocabulary for separating optimization variables, constraints, and validation requirements [7]. Topology Optimization (TO), including the SIMP method [8], remains a central baseline for load-path optimization [9]. Recent neural TO approaches [10] accelerate parts of this process. The present repository is closer to a generative prior over voxelized shapes than to classical topology optimization, but TO practice remains a useful reference for how strongly structural or performance claims should be validated.

Conventional aerodynamic shape optimization couples CFD solvers with numerical optimization over parameterized geometries, often using gradient or adjoint information [1], [11], [12]. Surrogate modeling is also widely used for efficient design-space exploration [2], [13]. In those settings, the geometry representation, optimizer, and solver are explicit components of a controlled loop. By contrast, the present repository samples from a learned latent model and then applies CFD-style evaluation. This makes the workflow exploratory, and success in code-path execution should not be confused with the stronger evidence expected in mature CFD-based shape optimization.

Physics-informed machine learning incorporates PDEs through PINNs [14], Neural Operators such as FNO [15], and operator-learning models such as DeepONet [16]. Differentiable-simulation work goes further by embedding differentiable fluid solvers into learning and design frameworks [17]. These directions represent stronger forms of "simulation in the loop" than the reduced sanity evidence reported here. The current repository uses an internal LBM-style scoring path as an implementation mechanism; using that path to promote labels across fidelity levels after validation remains future work [18].

Constraint-aware generation is another relevant line of work. Enforcing physical validity, including manufacturability and connectivity, remains a core challenge [19]. Oh et al. [20] demonstrated generative design for structures, while Steinmetz et al. [21] used differentiable projection for manufacturable designs. The current repository evidence is more limited: connectivity penalties, post-processing cleanup, and manufacturing-aware scoring proxies are useful for smoke-testing the code path, but they are not aircraft-specific validity guarantees.

The implementation also borrows standard architectural ingredients from modern deep learning, including attention-style modules inspired by the Transformer literature [22]. In this paper, those ingredients are treated as reused components rather than as a novelty claim.

A. Positioning and Novelty

The present repository does not claim a new diffusion objective, a new adjoint CFD method, or a validated aircraft-design benchmark. Its defensible contribution is narrower: a proof-of-concept assembly of a latent generative model, voxel decoding path, CFD-informed evaluation hook, STL export, and baseline and claim-gate checks in a single reproducible codebase. Relative to general 3D diffusion work, the emphasis here is on engineering-shaped voxel outputs rather than text-to-3D asset generation. Relative to topology optimization and classical CFD shape optimization, the repository explores a learned generative prior rather than directly optimizing a single parameterized shape. Relative to surrogate and differentiable-simulation papers, the current implementation is lighter-weight and presently supported only by sanity-run evidence.

This positioning is intentionally conservative. The repository should not yet be read as demonstrating superior exploration of an engineering design space, validated surrogate replacement, mission-conditioned aircraft generation, or manufacturing-conditioned aircraft generation. Those claims require the stronger evidence gates summarized below and in Section V.

B. Baseline and Ablation Protocol

The reduced final evidence package requires three executable baseline families: retrieval records from the grounded manifest, unconditional samples from the checkpoint, and bundled grounded STL examples. These are guardrail baselines: they verify that the report contains concrete comparison artifacts and repeated-run statistics, but they do not establish superiority over classical optimization or mature generative-model baselines.

Publication-grade baseline comparisons should still include four stronger families:

1) **Classical primitives:** hand-designed parametric aircraft primitives, such as swept or tapered wings extruded from NACA airfoil profiles, with controlled aspect-ratio, sweep, and taper ranges.
2) **Unconstrained generative 3D models:** representative voxel, point-cloud, and implicit-shape baselines such as voxel-GAN [23], point-cloud diffusion [24], and occupancy networks [25].
3) **Independent optimization search:** a SIMP-style topology-optimization comparison [8] and a random-search baseline in which sampled candidates are ranked by the same CFD evaluation protocol.
4) **Differentiable aerodynamic shape optimization:** an adjoint-based refinement baseline for a standard fuselage-wing primitive [12].

Ablations should isolate the impact of aerodynamic scoring diagnostics, connectivity diagnostics, the connected-component cleanup hook, the semantic constraint projector, and the consistency-step count. For now, that list is a protocol rather than a causal result; it becomes evidence only after larger grounded aircraft-like runs with shared solver settings.

III. Methodology

The current repository implements a proof-of-concept generative workflow built from four components: a noise scheduler, a latent diffusion-style denoiser, a latent-to-3D converter, and an internal CFD-oriented evaluator. The codebase now has two evidence tracks: deterministic synthetic/procedural smoke data retained for continuity, and a public VSP Airshow corpus used for the grounded training smoke run reported in Section IV. In this revision, the same path also carries a documented structured condition vector through dataset generation, latent construction, and checkpointed inference. Figure 1 should

TABLE I
EXPERIMENTAL PLAN AND CLAIM MAPPING BEYOND THE CURRENT GUARDRAIL BASELINES

Claim gated	Baseline or ablation	Metric family	Execution requirement
Aerodynamic efficiency	Classical primitives; adjoint ASO; no-CFD-loss ablation	C_L/C_D; drag coefficient; solver agreement	Shared geometry normalization, consistent reference area, and a documented CFD-resolution sweep.
Manifold integrity	Voxel-GAN; point-cloud diffusion; no-cleanup ablation	Connected-component ratio; repair success rate	Voxel-connectivity analysis on a fixed generated sample set and aircraft-specific geometric checks.
Computational efficiency	Random search with CFD ranking; long-step diffusion baseline	Inference latency; memory use; score stability	Wall-clock timing on declared hardware with fixed batch size and repeated runs.
Generative diversity	Parametric primitive library and conditioned batches	Shape similarity; latent coverage; part-volume variance	Grounded aircraft-like corpus, declared condition schema, and held-out condition-response tests.

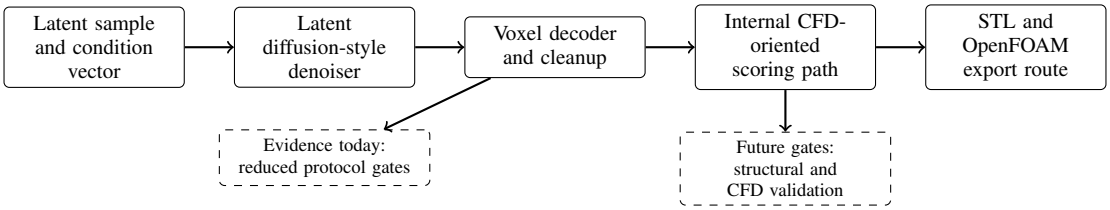


Fig. 1. Implementation-level view of the current repository. A latent code and optional structured condition vector are routed through denoising, voxel decoding, cleanup, internal scoring, and export paths. The diagram describes the code path under test; it does not by itself establish differentiable training, structural viability, or publication-grade CFD validation.

therefore be read as an implementation diagram for the present repository rather than as evidence of a fully validated aircraft-design system.

A. Public Airshow Corpus Construction

The grounded corpus path collects public model documents from the VSP Airshow web app, keeps only records whose Airshow license identifiers map to CC0, CC BY, or CC BY-SA, and requires a public preview-geometry URL before a record can enter the training manifest [26]–[28]. The builder parses available X3D indexed-face-set geometry, normalizes each mesh into a centered unit cube, voxelizes the occupied geometry into a fixed grid, and records source URLs, license metadata, geometry hashes, voxel hashes, split assignment, and preprocessing provenance. The primary smoke run uses a 16^3 grid, and the later resolution addendum reruns the same corpus construction at 32^3, 64^3, and 96^3. The deterministic split assignment is stored in the manifest so that later validation, training, and generation commands can be tied back to the same corpus lineage.

The run used in this paper observed 381 public Airshow model documents, admitted 357 license-and-geometry-eligible documents, and converted 355 records after two public storage URLs returned 404. Figure 2 visualizes that corpus funnel together with the license and split distributions. Airshow model names, manufacturer fields, URLs, and license labels are treated as source metadata only. The additional mission and manufacturing fields used by the condition vector are deterministic repository inferences from geometry and defaults, not factual claims made by NASA, Lockheed, OpenVSP, or any other named source.

B. Noise Scheduling

The forward diffusion process gradually adds Gaussian noise to the input data over a series of T timesteps. The noise schedule is defined by a variance schedule β_t, which is typically a linear or cosine schedule. The implementation uses a linear schedule, where β_t increases linearly from β_{start} to β_{end}.

The forward process is defined as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{1}$$

During reverse sampling, the denoiser estimates the noise component of a noisy latent. The training loss compares predicted noise with sampled noise; sampling then uses that estimate to update the latent.

C. Latent Diffusion UNet

The denoising model is a UNet-based architecture that operates in the latent space. The UNet takes as input a noisy latent vector, a timestep embedding, and an optional condition embedding, and outputs the predicted noise. The implementation projects the latent to a small spatial tensor and applies residual block stacks with additive skip connections. The skip path is intended to preserve intermediate spatial features during denoising.

D. Latent-to-3D Converter

The latent-to-3D converter maps the denoised latent vector to voxel logits. For lower-resolution smoke runs, this is a dense multi-layer perceptron (MLP) whose final layer emits the whole voxel grid. For the 96^3 follow-up, the implementation switches to a coordinate decoder that evaluates an MLP over

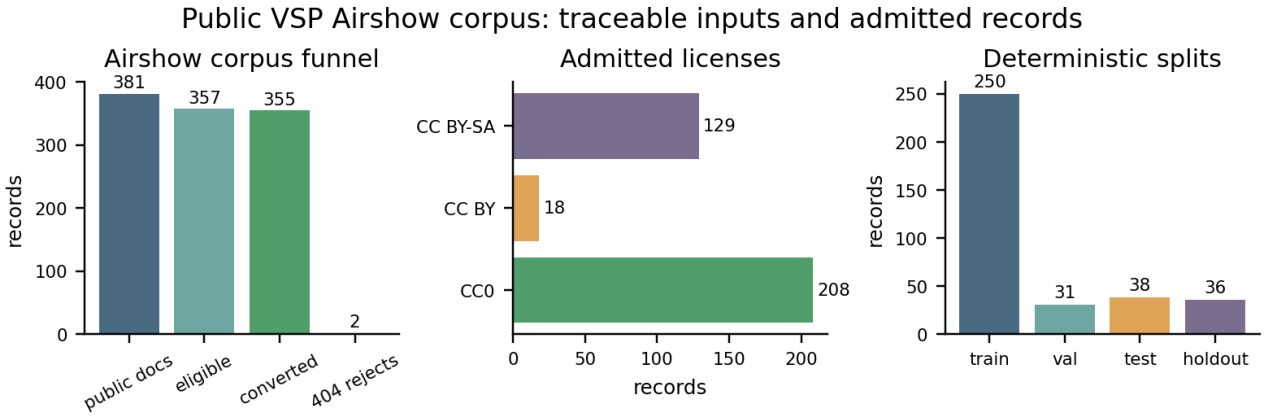


Fig. 2. Corpus construction summary for the public VSP Airshow smoke run. The plot shows the observed Airshow model-document count, license-and-geometry filtering, converted voxel records, admitted license mix, and deterministic train/validation/test/holdout split. These counts document data provenance and coverage; they do not certify manufacturer approval or aircraft performance.

the latent vector concatenated with normalized (z, y, x) coordinates; training samples voxel coordinates with importance-weighted BCE so occupied coordinates can be oversampled without changing the intended sparse reconstruction objective. Downstream training and generation code applies a sigmoid to convert logits into voxel probabilities.

E. CFD Simulator

The repository uses an internal lattice-Boltzmann-style CFD evaluator to score generated voxel grids. The evaluator takes a voxel geometry and flow settings such as Mach number or Reynolds number and returns drag- and lift-related quantities. In addition, the repository can export benchmark cases for comparison against an external OpenFOAM run. This combination is useful for code-path validation, but the present paper does not treat it as a substitute for a full aerodynamic validation campaign.

F. Loss Function

A simplified view of the current training objective distinguishes the base differentiable model losses from the measured solver-in-loop term. The optimizer step uses diffusion, decoded-geometry reconstruction, direct generation reconstruction, consistency, and, when scheduled, a direct internal-solver objective:

$$\mathcal{L}_{opt} = \mathcal{L}_{diff} + \lambda_{geom}\mathcal{L}_{geom} + \lambda_{gen}\mathcal{L}_{gen} + \mathcal{L}_{cons} + \lambda_{solver}\mathcal{L}_{solver\text{-}SPSA}. \tag{2}$$

The diffusion loss is the mean squared error between the predicted noise and the actual noise. The decoded-geometry and generation-reconstruction terms compare voxel logits against the target voxel grid, while the consistency term supports the reduced-step sampler. The solver term materializes voxel probabilities into a full grid, converts them to a binary geometry by thresholding or top-k occupancy selection, calls the internal D3Q27 lattice-Boltzmann evaluator on that geometry, and combines the measured aerodynamic score with an optional exact connected-component penalty. This path is not a learned stand-in: the scalar used in $\mathcal{L}_{solver\text{-}SPSA}$ comes from solver calls made during training.

The connected-component operation, binary materialization step, and internal CFD evaluator are not ordinary PyTorch operations with analytic gradients. The training loop therefore uses a two-sided simultaneous perturbation estimate when this term is active. For a probability grid V, a perturbation field Δ, and perturbation scale ϵ, it evaluates the measured objective on $V + \epsilon\Delta$ and $V - \epsilon\Delta$ and uses

$$\widehat{\nabla}_V \mathcal{L}_{solver} \approx \frac{\mathcal{J}(V + \epsilon\Delta) - \mathcal{J}(V - \epsilon\Delta)}{2\epsilon}\Delta \tag{3}$$

as the backward signal. The implementation can draw Δ directly at voxel resolution or on a lower-frequency grid that is upsampled to the solver lattice before evaluation. The latter reduces high-frequency finite-difference noise and keeps the expensive solver work sequential and scheduled rather than evaluating many independent CFD calls in parallel.

The exact connected-component score and raw internal solver score can still be run as measured monitors,

$$\mathcal{D}_{total} = \mathcal{L}_{opt} + \mathcal{D}_{conn} + \mathcal{D}_{aero}, \tag{4}$$

where $\mathcal{D}_{conn}$ is a connected-component score on thresholded voxels and $\mathcal{D}_{aero}$ is an internal solver score on thresholded generated geometry. In the direct-solver runs, those detached monitor intervals are set to zero so the limited full-grid compute is spent on the solver term that contributes to $\mathcal{L}_{opt}$. The reported `direct_solver_loss` is the optimizer-facing term averaged over all training batches, and `direct_solver_eval_loss` is the mean over the scheduled solver evaluations.

G. Sequential Measured-Objective Candidate Optimization

The repository now includes an experimental sequential optimizer for generated candidates. This path treats the connectivity, validity, and aerodynamic evaluations as measured black-box objective terms:

$$\mathcal{L}_{seq} = w_{conn}\mathcal{D}_{conn} + w_{aero}\mathcal{D}_{aero} + w_{valid}\mathcal{D}_{valid} + w_{occ}\mathcal{D}_{occ}. \quad (5)$$

The implementation supports a sequential genetic search and a two-point SPSA-style estimator. Both methods evaluate one candidate at a time, record the measured objective terms, and keep or update candidates based on the scalar score. This is a real optimizer over generated candidates, but it is not backpropagation through the CFD solver and it does not by itself update the diffusion model weights.

H. Current CFD Coupling In The Training Loop

The codebase is designed to couple geometry generation and CFD-informed scoring during training. In the present implementation, that coupling has two honest forms: direct black-box solver optimization inside the training loop and measured black-box candidate optimization after generation. The raw internal LBM evaluator remains sequential and non-differentiable in the analytic-autograd sense, but its scalar outputs can still influence model weights through the SPSA estimator described above.

In the current loop, training starts by sampling a noisy latent z_t, asking the UNet for a noise estimate or denoised latent estimate, and decoding the resulting z_0 candidate into voxel probabilities V. For high-resolution coordinate decoders, the loop normally trains on sampled coordinates and only materializes the full grid when a full-grid objective is scheduled. When $\mathcal{L}_{solver\text{-}SPSA}$ is active, the loop materializes that grid, evaluates the internal D3Q27 LBM objective on the base geometry and on two perturbed geometries, and folds the resulting measured scalar into the same `optimization_loss` that is backpropagated through the decoder and denoiser weights.

The OpenFOAM setup is the external PDE-validation foundation. Its role is to generate independent reference cases and, when residual and field-export gates pass, promote labels to the external-PDE tier defined in the ground-truth specification. The present Airshow grid-loss sweep uses the internal D3Q27 solver as a measured training objective and does not use OpenFOAM labels, so it should be read as internal-LBM-guided optimization plus a validation route, not as externally validated aerodynamic learning.

I. Monitored Training and Recovery Infrastructure

The production training path is an interruption-safe monitored runner rather than a bare training loop. It maintains durable run-state checkpoints on a fixed cadence, and each checkpoint records the model weights (diffusion, consistency, converter, and EMA), optimizer and scheduler state, RNG state, the frozen geometry-materialization threshold, and a compatibility fingerprint covering the model, diffusion, and training configuration. Writes are atomic: the runner writes to a temporary file, fsyncs, and atomically replaces the destination so a crash cannot leave a torn checkpoint. The recorded fingerprint is used at resume time to fail closed if the current configuration would reconstruct a differently-shaped model, instead of silently loading mismatched weights.

Two recovery modes are supported on top of this state. An exact resume restores the full optimization lineage (step counters, optimizer moments, RNG, split/sample order, threshold calibration) from a run-state checkpoint, so an interrupted run can continue deterministically. A warm start instead treats the checkpoint as weights-only lineage: it reconstructs the saved architecture from the checkpoint metadata, loads compatible weights while allowing older checkpoints to omit opt-in keys, resets the optimization lineage, and re-enters a bounded warm-up window. Warm starts are gated by grid-resolution and latent-width checks against the requested run, and recovery-mode runs lower the collapse-watchdog warm-up so the run is quickly re-supervised.

The collapse watchdog supervises every monitored run with fail-closed numerical and geometric guards. It monitors the probability standard deviation and span floor, occupied-fraction bounds, converter gradient norms and update lag, unique-sample fraction, and relative (ratio-based) floors so a silent collapse on any single trace does not go unnoticed. On repeated violations it stops the run with a machine-readable diagnostic. This infrastructure is what makes the longer 128^3 final protocol executable: the final run can be checkpointed every 25 updates, resumed exactly after interruption, and stopped promptly if the optimizer collapses, so a single unattended run does not depend on the machine staying up.

The final-run protocol (Section IV-K) targets the scaled architecture selected by the capacity law: 128^3 grid, 512-dim latent, coordinate decoder of width 2048 and depth 5 (70.1M trainable parameters) in bfloat16 on a 24 GB-class GPU, with a fused-stream BFL solver backend and a pinned-latent training corpus. All architecture values are asserted by a protocol-side manifest and by the automated test suite, so a launch that silently uses different widths fails before burning GPU budget.

IV. Results and Discussion

The empirical evidence in this revision is intentionally narrow. It consists of public-geometry smoke training on VSP Airshow models, a higher-resolution Airshow addendum, bounded smoke runs on synthetic or procedural data retained for context, a reduced freeform-object sweep for the historical CFD path, and smoke-level checks for the new structured-conditioning seam. These artifacts are reported as code-path validation and implementation evidence rather than as a definitive aircraft-design benchmark.

A. Reduced Training Smoke Evidence

Figure 3 summarizes the scope of the reduced training evidence retained for context. Its purpose is to show which code paths were exercised in bounded smoke runs. It should not be read as a convergence study or as evidence that the current repository has validated aerodynamic or structural learning behavior.

B. Public Airshow Corpus Training Smoke Evidence

To replace the earlier synthetic-only scale-up evidence, we built a public VSP Airshow corpus from model documents

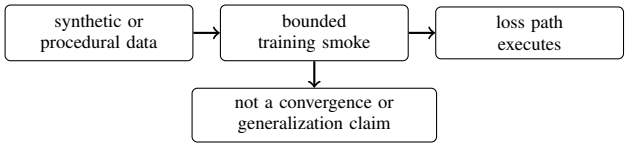


Fig. 3. Evidence scope for the reduced training runs. The current paper treats these runs as implementation evidence only, not as a full convergence study.

TABLE II
PUBLIC AIRSHOW CORPUS SMOKE RUN

Field	Value
Converted records	355
Split counts	train 250, val 31, test 38, holdout 36
Licenses	208 CC0, 18 CC BY, 129 CC BY-SA
Manifest hash	`7bb59bab9cc8...451885a6`
Checkpoint hash	`71e808aa3c35...6413d4f6`
Training scope	3 epochs, batch 8, latent 16, D3Q27, global step 135

TABLE III
GENERATED FLIGHT-PATH SMOKE CHECKS FROM THE AIRSHOW CHECKPOINT

Case	Occ.	C_d	L/D	Validity
short takeoff payload	0.01416	1.2047	0.00438	fail: span sanity
high-speed sprint	0.01294	0.9122	0.03051	fail: span sanity
endurance turning	0.01245	0.9824	0.04663	fail: span sanity

exposed by the Airshow web app. OpenVSP describes itself as a parametric aircraft geometry tool, and the OpenVSP project site links VSP Airshow as a public model exchange [26]–[28]. The corpus builder admits only CC0, CC BY, and CC BY-SA Airshow entries with public preview geometry; it excludes NC and ND entries from the training manifest. In the run reported here, the builder observed 381 public model documents, found 357 license-and-geometry-eligible documents, and converted 355 records into centered 16^3 voxel grids. The two rejected eligible documents had stale public storage URLs returning 404.

The last epoch's terminal output is kept in the paper because it is the run record: diagnostic total 21.5905, MSE 0.7997, geometry-reconstruction loss 0.0778, consistency loss 0.00109, connectivity diagnostic 0.00149, and aerodynamic diagnostic 20.7104. These values are smoke-run diagnostics only, not convergence evidence. A later loss-semantics audit showed that the connectivity and aerodynamic quantities in this implementation are detached diagnostics rather than differentiable solver-training signals. Figure 4 plots the same three-epoch history so the increasing diagnostic total and aerodynamic diagnostic remain visible rather than implied away. Airshow names, manufacturer fields, licenses, and URLs are treated as source metadata; the generated mission and manufacturing fields in the manifest are deterministic conditioning inferences, not facts asserted by the named manufacturers or agencies.

All three generated cases produced nonempty voxel grids, STL exports, finite internal D3Q27 coefficients, and positive reference areas. However, all three failed the current aircraft-

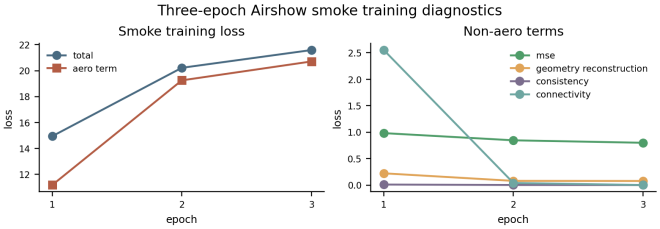


Fig. 4. Terminal-observed three-epoch Airshow smoke-training diagnostics. The plot is included to document the run, including the rising diagnostic total and aerodynamic diagnostic; it is not interpreted as convergence evidence.

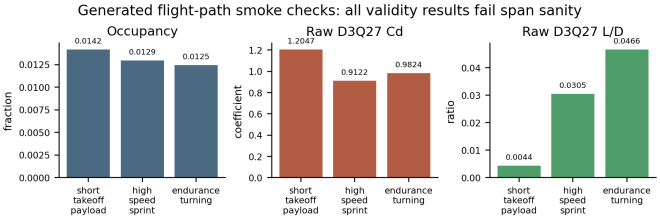


Fig. 5. Raw generated flight-path smoke metrics from the Airshow checkpoint. The occupancy, C_d, and L/D values are finite internal D3Q27 outputs, but every case fails `span_sanity` and the solver labels remain non-claim-bearing.

specific validity screen on the span-sanity check, and the CFD report marks the D3Q27 outputs as non-claim-bearing raw LBM labels. Figures 5 and 6 show the raw metric spread and the generated voxel geometry/projection views. We therefore interpret this run as evidence that the public-corpus training, generator, export, heuristic-screen, and solver paths execute together; it is not evidence that the smoke checkpoint generates valid aircraft or validated aerodynamic predictions.

C. Airshow Resolution Sweep and Loss Debugging

We also reran the Airshow corpus path at 32^3, 64^3, and 96^3 to check whether higher voxel count alone would clear the generated-aircraft validity gate. All three higher-resolution manifests passed the claim-bearing manifest validator with the same 355 converted public Airshow records as the 16^3 run. The 32^3 training run completed one epoch with batch size 2 and produced checkpoint hash `7234e1b9b338...b95444`. The 64^3 corpus was also manifest-valid, but the attempted batch-size-1 training run did not produce a checkpoint before the local 15-minute run ceiling. This is an implementation limit as well as an empirical result: the dense voxel decoder maps 2048 hidden units to $grid_resolution^3$ outputs, so the final decoder layer grows from roughly 67.1 million parameters at 32^3 to 537.1 million at 64^3 and 1.812 billion at 96^3.

The 32^3 manifest hash was `7684e70b...98521f1`, the 64^3 manifest hash was `2627227f...4a1dbd80`, and the 96^3 manifest hash was `86f2a112...118e7cd`. The 32^3 generated cases are shown in Figures 8 and 9. Their occupancy fractions were 0.00494, 0.00476, and 0.00510. Their raw internal D3Q27 C_d values were finite, but all three failed the aircraft-validity screen: two failed `nonempty_occupancy`, `symmetry`, and `span_sanity`, while the third failed

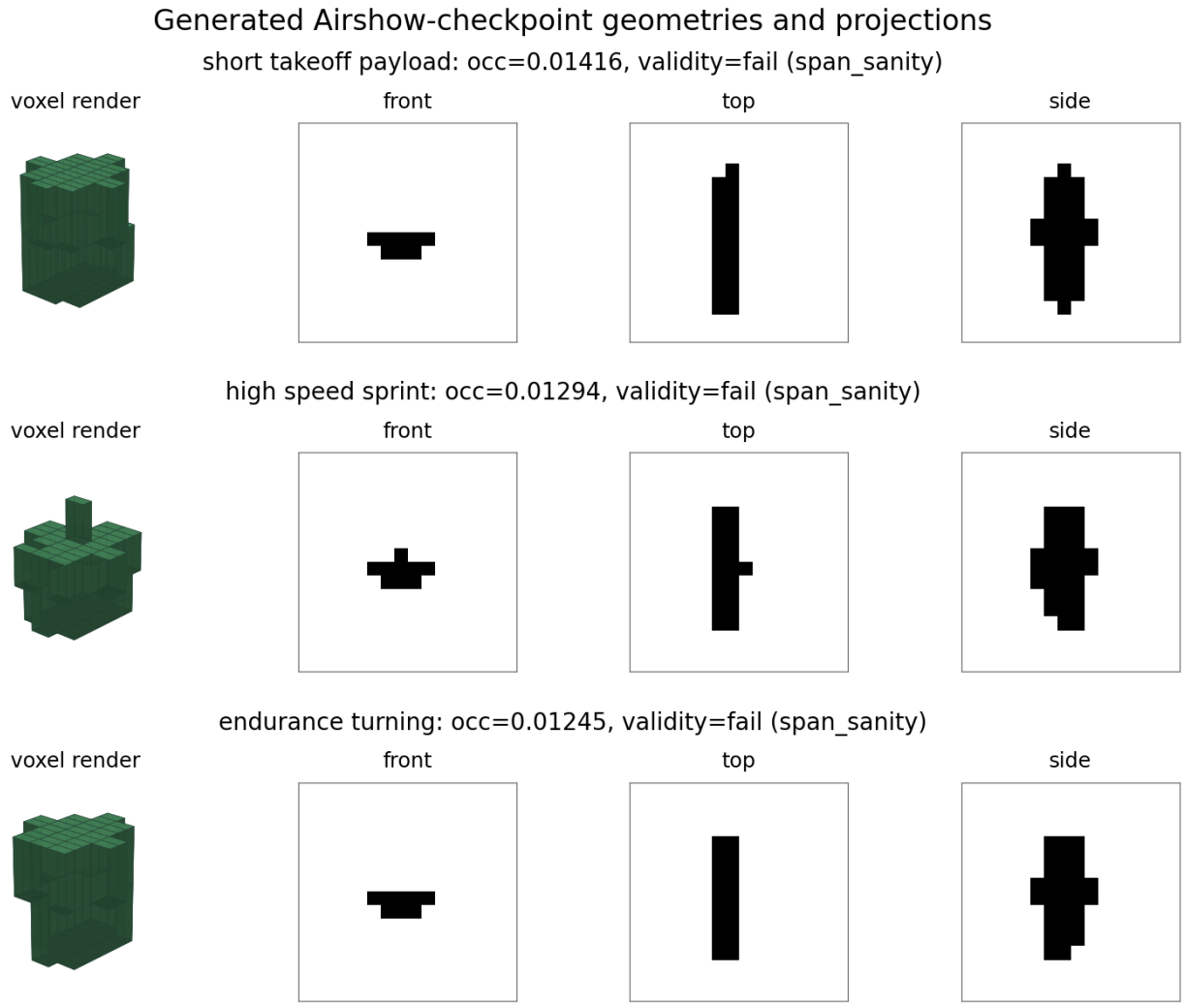


Fig. 6. Generated Airshow-checkpoint voxel geometries and orthographic occupancy projections for the three conditioned smoke cases. The visualized artifacts are intentionally shown despite failing `span_sanity`: they demonstrate nonempty generation and exportable grids, while also showing why the current checkpoint should not be described as producing valid aircraft.

TABLE IV
AIRSHOW RESOLUTION SWEEP ADDENDUM

Grid	**Manifest**	**Training outcome**	**Generated gate outcome**
32^3	pass, 355 records	one epoch completed on CUDA, batch 2	three generated cases failed validity
64^3	pass, 355 records	attempted batch 1; no checkpoint produced in local run ceiling	not run because no checkpoint existed
32^3, source-valid filter	pass, 176 records	three-epoch LR-fixed fine-tune completed from the 32^3 checkpoint	three generated cases failed only `span_sanity`
96^3, coordinate decoder	pass, 355 records	three epochs completed on CUDA, batch 1, importance-weighted coordinate BCE	fixed-threshold exports failed; calibrated top-k export passed 1 of 3 heuristic screens

`symmetry` and `span_sanity`. The result is therefore negative evidence against treating resolution alone as enough to make the present checkpoint claim-bearing.

We then filtered the 32^3 corpus with the same aircraft-validity screen used for generated outputs. This kept 176 of 355 public Airshow records and produced a claim-bearing-valid manifest hash `0d149d98...d25ccc`. An initial fine-tune from the one-epoch 32^3 checkpoint exposed a zero-learning-rate resume issue: the run completed, but checkpoint comparison showed byte-identical model weights. After restoring the configured learning rate on resume, a three-epoch source-valid fine-tune produced checkpoint hash `657243ea...45574c`. Its three generated cases all passed occupancy and symmetry but still failed `span_sanity`, with length-fraction values 0.25, 0.25, and 0.28125. A further three-epoch continuation regressed validity, with two cases failing both `symmetry` and `span_sanity`. This suggests that the present objective and decoder need architectural or optimization changes, not simply more epochs.

The 96^3 follow-up replaced the dense decoder with a coordinate decoder that evaluates sampled (z, y, x) coordinates

conditioned on the latent vector. The public 96^3 source corpus had mean occupancy 0.00565 and median occupancy 0.00458, so naive coordinate sampling either collapsed toward empty fixed-threshold grids or, when occupied coordinates were oversampled without correction, biased the decoder toward overly dense outputs. The final run used importance-weighted coordinate BCE for three epochs over all 355 records and produced checkpoint hash `1bfdcfcf...59774d3`. With the default 0.5 threshold, all three flight-path exports remained below the aircraft-validity occupancy gate. With an explicit top-k export at 0.75% occupancy, one case, `endurance_turning`, passed the heuristic aircraft-validity screen; `short_takeoff_payload` and `high_speed_sprint` still failed `symmetry`, `span_sanity`, and `wing_body_balance`. Figures 10 and 11 show that calibrated export result. This is evidence that the coordinate-decoder path can train and export 96^3 geometry, not evidence of reliable aircraft generation or validated aerodynamic optimization.

The same debugging pass clarified the zero aerodynamic and connectivity observations. The exact connectivity score thresholds voxel probabilities and runs connected-component labeling through NumPy/SciPy. The raw aerodynamic score thresholds geometry, invokes the internal solver, and returns scalar solver outputs. A local gradient probe confirmed that these measured scores do not provide analytic PyTorch gradients by themselves. The training code was therefore revised to report `optimization_loss` separately from detached monitor terms and to add a direct solver-in-loop loss that calls the internal D3Q27 LBM evaluator during training and estimates its backward signal with two-sided SPSA finite differences. This is a measured black-box gradient estimator around real solver calls, not a learned substitute for the solver.

The first direct-solver sweep was intentionally diagnostic. A naive 0.5 probability threshold produced empty or effectively empty scheduled geometries, so the direct solver objective stayed nearly constant. A 5% top-k occupancy target then made the solver-visible shapes too dense relative to the public Airshow corpus, whose mean occupancy was 0.01495 at 32^3, 0.00802 at 64^3, and 0.00565 at 96^3. The final sweep therefore used a 1% top-k target, a low-frequency 8^3 perturbation field upsampled to the solver grid, a direct-solver loss weight of 0.2, three D3Q27 solver steps per scheduled objective call, and zero detached aerodynamic/connectivity monitor intervals. That setting spends the full-grid compute on the loss being optimized rather than on diagnostics that cannot change the weights.

Table V shows both the matched two-epoch sweep and the required 96^3 continuation. After two epochs, the 96^3 optimizer loss and scheduled solver evaluation were still worse than the 64^3 run even though its geometry BCE was lower. Continuing from the 96^3 checkpoint corrected that behavior: the scheduled direct solver evaluation dropped from 180.29967 to 3.87060 on the first continuation epoch and remained much lower than the 64^3 value at 7.48613 on the second continuation epoch. The increase from 3.87060 to 7.48613 while the optimizer loss continued to decrease is a useful warning that the direct objective can start to overfit or drift under this short local

TABLE V
DIRECT SOLVER-IN-LOOP GRID SWEEP

Run	Opt. loss	Geom. BCE	Direct loss	Scheduled eval
32^3, epoch 2	2.10166	0.08267	5.83028	172.47912
64^3, epoch 2	1.11359	0.05038	1.30224	38.52468
96^3, epoch 2	2.05984	0.04066	6.09464	180.29967
96^3, continuation epoch 1	0.68305	0.03802	0.13084	3.87060
96^3, continuation epoch 2	0.60605	0.03739	0.25305	7.48613

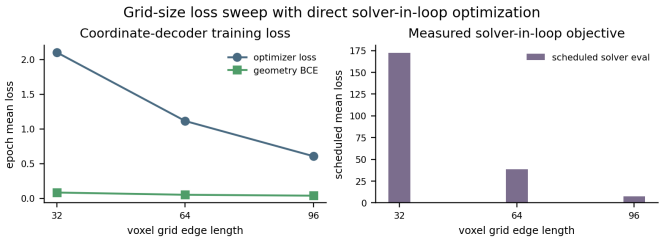


Fig. 7. Grid-size loss sweep with the coordinate decoder forced at 32^3, 64^3, and 96^3. The plotted 96^3 point uses the continuation run because the matched two-epoch 96^3 loss was still worse than 64^3. The right panel reports the scheduled measured D3Q27 solver objective used by the SPSA loss; it is internal solver-in-loop evidence, not external PDE validation.

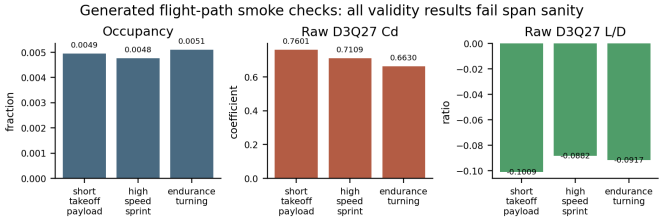


Fig. 8. Raw 32^3 generated flight-path metrics. The generated artifacts have finite internal D3Q27 outputs, but all still fail aircraft-validity gates and therefore remain non-claim-bearing.

protocol. Figure 7 therefore uses the final continuation point for the 96^3 visual trend while the table preserves the non-monotone intermediate result.

D. Freeform-object Sweep: Steps vs. Shape Prior

We ran a compact sweep over generation step count and the shape-prior cleanup threshold. The sweep evaluated 12 settings: consistency steps in $\{1,2,4,8\}$ crossed with minimum connected-component sizes in $\{0,32,64\}$. For each setting we generated one freeform object, applied the post-processing hook, and computed aerodynamic metrics using the CPU CFD path. The sweep provides only a preliminary probe of how post-processing thresholds can alter the generated object before scoring; Figure 12 documents the reduced coverage.

E. Freeform-object Geometry Summary

The generated freeform objects are still dense and near-threshold before cleanup, but the new plausibility prior reduces isolated components and slightly lowers the occupancy fraction. For this implementation path, that matters more than

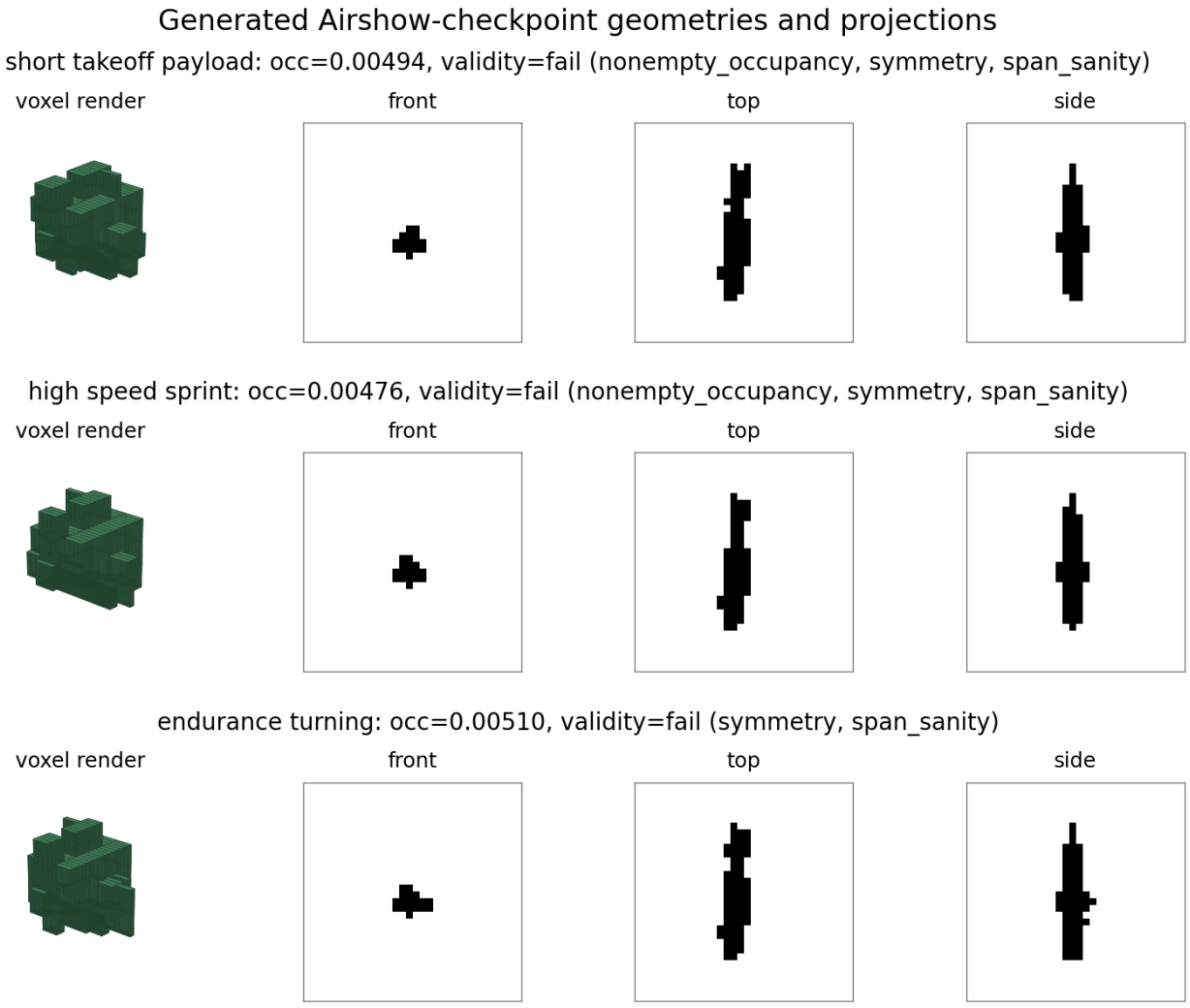


Fig. 9. 32^3 generated Airshow-checkpoint voxel geometries and projections. The denser grid increases visual resolution but does not resolve the span-sanity and symmetry failures that block aircraft-generation claims.

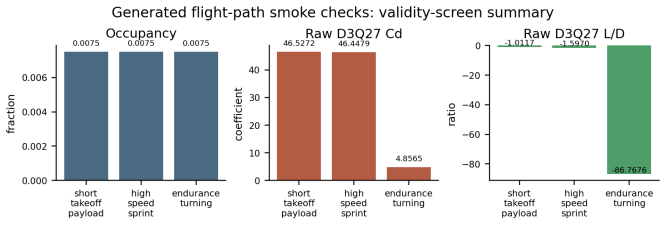


Fig. 10. Calibrated 96^3 top-k flight-path metrics at 0.75% occupancy. One case passes the heuristic aircraft-validity screen, while two still fail shape checks; all CFD quantities remain internal non-claim-bearing D3Q27 smoke outputs.

small changes in the raw voxel logits: the cleaned binary shapes are less fragmented and more suitable for downstream meshing and export. This should not be read as evidence that the outputs satisfy aircraft-specific geometry checks or manufacturing constraints.

F. Hyperparameter and Tuning Notes

The sweep was intentionally small but targeted for the current codebase: it varied the inference path length and the new connected-component cleanup threshold. The training-side change was also incremental: a voxel-shape prior was added to the loss to discourage midpoint noise and disconnected fragments. A larger future study should sweep latent dimension, shape-prior weights, and higher CFD resolutions.

G. Structured-Conditioning Smoke Evidence

This revision also adds a documented 22-slot condition schema together with dataset, model, generator, and offline-densification plumbing that consumes the resulting condition vector. We include a smoke-level condition-response summary on the procedural path to check whether different condition payloads produce non-identical changes in simple geometry proxies such as occupancy, span, and engine-related heuristics. That evidence is intentionally weak: it shows that the condition path is wired and that the procedural prior is not condition-invariant, but it does not validate mission-conditioned or

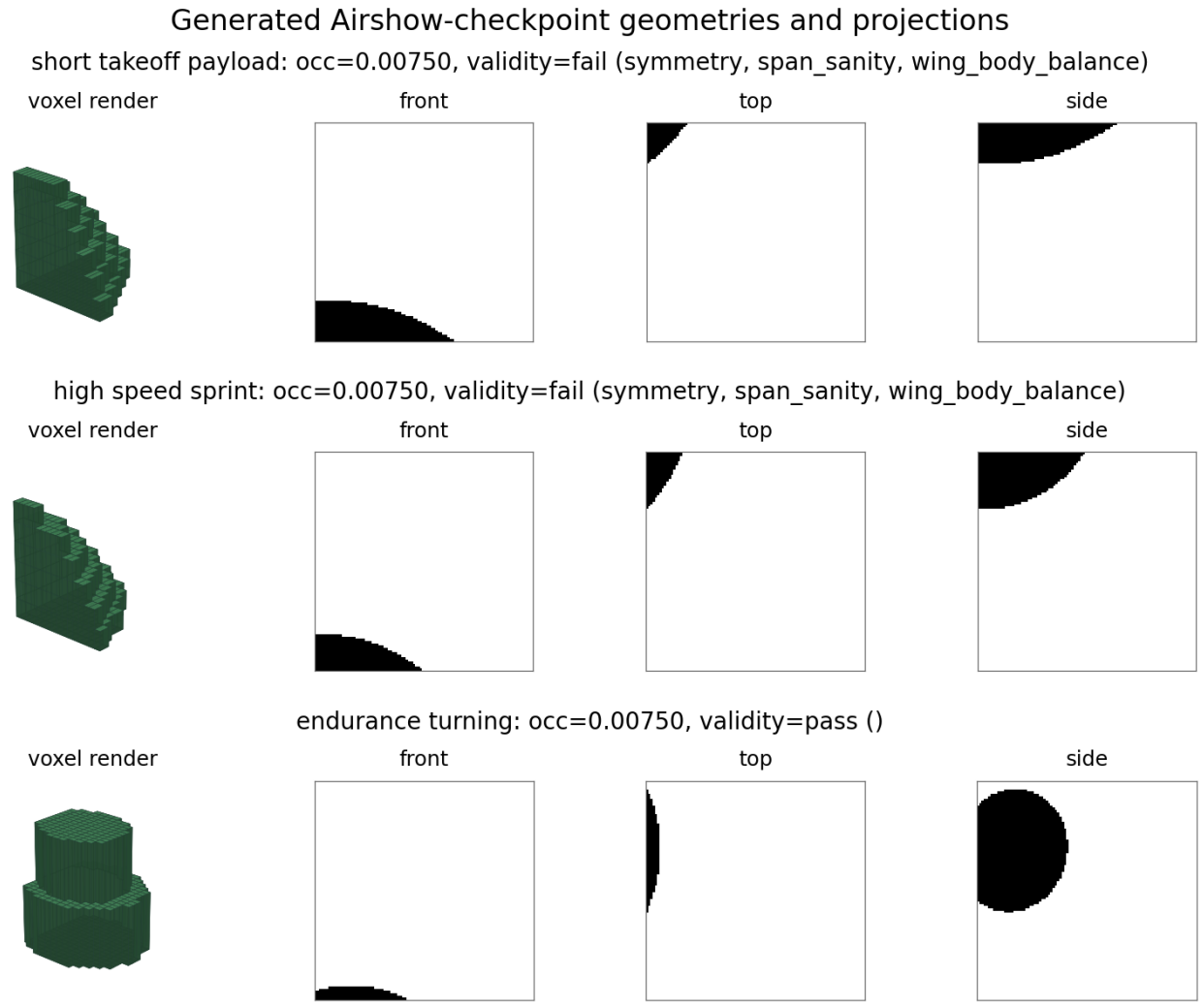


Fig. 11. Calibrated 96^3 coordinate-decoder geometries and projections. The figure documents the best high-resolution smoke result, including the two failing cases, rather than upgrading the result to a reliable aircraft-generation claim.

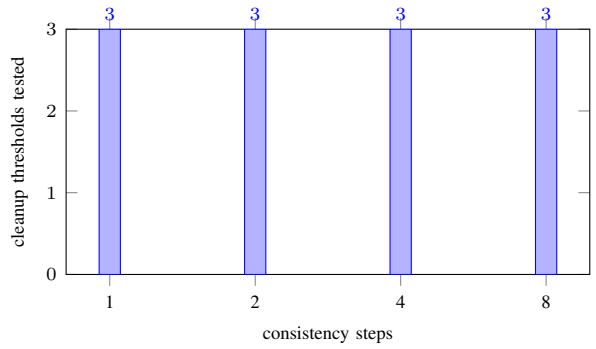


Fig. 12. Reduced sweep coverage used for code-path probing: four consistency-step settings crossed with three cleanup thresholds. Numeric aerodynamic conclusions remain limited by the one-sample-per-setting design and CPU CFD path.

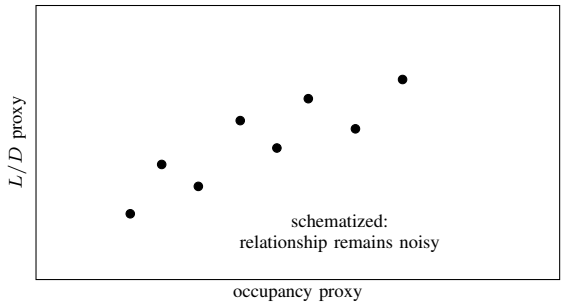


Fig. 13. Interpretive schematic for the occupancy-versus-score question raised by the reduced sweep. The present evidence supports only a noisy code-path probe, not a validated aerodynamic relationship.

manufacturing-conditioned aircraft generation on grounded aircraft-like data.

H. Reduced Evidence Package

The checked-in reduced protocol assembles a package-level evidence report from the manifest validator, aircraft-shape heuristic screen, grounded condition-response benchmark, manufacturing-bounds screen, and baseline-statistics

TABLE VI
REDUCED EVIDENCE PACKAGE STATUS

Gate	Status	Role
Manifest validation	Pass	corpus contract check
Aircraft-shape screen	Pass	heuristic generated-shape checks
Condition response	Pass	fixed grounded sweeps
Manufacturing bounds	Pass	design-spec bounds check
Baseline statistics	Pass	guardrail baselines and finite-value seed checks

TABLE VII
INTERNAL SOLVER C_d SANITY TARGETS AND OBSERVATIONS

Geometry	Re	Observed C_d	Literature target
Circular cylinder	100	1.56	1.3–1.4 [29]
Unit cube	1000	1.04	∼1.05 [30]

report. In one local protocol run, all package gates passed with a shared run identifier, checkpoint hash, manifest hash, and protocol hash. Table VI summarizes the package status.

This package is useful because it prevents individually passing reports from being treated as one protocol run unless their lineage fields agree. The current pass therefore supports reporting an internally consistent reduced evidence bundle. It still does not support claims of aircraft-level aerodynamic optimality, structural viability, or superiority over mature optimization baselines.

I. Internal Solver Validation against Canonical Benchmarks

As a solver sanity check, we compare internal D3Q27 drag-coefficient (C_d) observations against literature targets for two standard geometries: a circular cylinder and a unit cube. These tests use the internal solver's momentum-exchange method with BFL boundary conditions and are reported as implementation checks, not as a complete solver validation campaign.

The results in Table VII are an informal sanity check for the LBM implementation, not a full solver validation. The domain spans ten characteristic lengths in the streamwise direction and five characteristic lengths in each cross-stream direction, with 128 cells per axis. Boundary conditions are no-slip BFL walls and a Neumann outlet. The inlet speed corresponds to $Ma = 0.025$, or approximately 0.014 in lattice units, and the force estimate is averaged over the final 125 steps of a 500-step simulation to reduce transient sensitivity.

After correcting the freestream lattice-velocity initialization and the BFL momentum-exchange weighting term, the observed C_d values sit close to or inside the broad range of the cited targets. Because exact 3D benchmarks depend on blockage ratio and averaging policy, these results support continued use as a bounded implementation check; rigorous validation still requires comparison with high-fidelity PDE solvers.

J. Discussion

The implementation evidence supports three bounded conclusions: the repository's end-to-end path executes, the shape prior is wired into training, and the export path emits a concrete OpenFOAM comparison case with the generated STL in `constant/triSurface/design.stl`. On the same reduced cube benchmark, the internal D3Q27 solver and the OpenFOAM sonicFoam run both complete on the shared validation geometry, providing a direct solver-to-solver comparison for the current pipeline. The result supports a documented implementation path, not a claim of aircraft-level aerodynamic optimization.

The OpenFOAM export path now writes a plain `forces` function-object dictionary, and the resulting force vector is normalized manually to recover coefficients when coefficient output is not available. The current benchmark still shows a substantial Cd gap, which we treat as a solver- and sampling-consistency diagnostic rather than as resolved by paper-level normalization alone. A higher-confidence CFD comparison will still need a more systematic hyperparameter sweep and consistent normalization of reference area and flow conditions.

The current ground-truth foundation is therefore tiered. Internal D3Q27 LBM values can drive the direct SPSA training loss and sequential candidate selection, but they stay in the internal-label tier. The OpenFOAM/simpleFoam and sonicFoam scripts define the external PDE route: cases must complete, residual and field-export checks must pass, and the resulting labels must remain separate from internal LBM labels until they satisfy the `external_pde` gate. This separation is important for future PDE validation because it lets the repository compare internal LBM coefficients against OpenFOAM under shared geometry, Mach, Reynolds, grid, and reference-area settings without silently upgrading the internal solver to ground truth.

We also recorded one physics correction in `CLI/cascaded_lbm.py`. The D3Q27 freestream setup now uses the lattice-consistent value $u = Ma/\sqrt{3}$; the earlier version used arbitrary scaling. That correction is part of the benchmark history, and we are checking it against the Cd mismatch before making broader coordinate-transform changes.

K. The Single Decisive Final Run

The checked-in final protocol (`CLI/run_protocols/gcp_128_295m.yaml`, executed by the monitored runner) is deliberately the single experiment this paper designates as the gate for its remaining aircraft-level claims. It trains the scaled 128^3 architecture (512-dim latent, coordinate decoder width 2048 depth 5, 70.1M trainable parameters) in bfloat16 for two epochs at batch size 1 on the pinned-latent training corpus, checkpointing every 25 updates under the interruption-safe runner, with the fused-stream BFL solver backend and a GPU-side exact-SDF attestation so the reported geometry pipeline is the measured one. Because the run is monitored, it produces the run-state lineage (checkpoint hashes, manifest

TABLE VIII
CLAIM GATE STATUS AND FINAL-RUN UPGRADE CONTRACT

Claim	Current status	Final run unlocks if
Generates aircraft structures	blocked	generated samples pass validity screen at declared rate
Conditioned on mission/manufacturing	blocked	grounded condition benchmark shows directional held-out signal
CFD-guided training	blocked	scheduled direct SPSA objective decreases without drift
Aerodynamically optimized	blocked	requires separate converged CFD vs. baselines (not this run)
Structurally viable	blocked	requires separate structural/load-path analysis
Superiority over prior methods	blocked	requires separate baseline comparison studies
Compressible internal solver validated	blocked	requires separate external compressible validation

hashes, protocol hashes) needed by the evidence package rather than an unrepeatable training history.

Success for this run is defined by four gates, all of which are executable in the repository today:

1) **Generated-sample validity:** generated flight-path conditioning cases pass the aircraft-specific validity screen (currently `span_sanity`, symmetry, planform-sparsity, and fuselage-end checks) at a declared rate.
2) **Condition-response signal:** the grounded condition benchmark shows directional signal on held-out records rather than condition-invariant generation.
3) **Solver-in-loop signal:** the scheduled direct D3Q27 SPSA objective decreases over the run instead of drifting, and the evidence package records it.
4) **Lineage-consistent evidence:** `CLI/final_evidence.py` returns `pass` with matching run/checkpoint/manifest/protocol hashes.

This one run is the paper's upgrade contract. If it passes gates 1 and 2, the claim rows `Generates aircraft structures` and `Conditioned on flight profile and manufacturing method` in Table VIII move from blocked to supported; if gate 3 passes, `CFD-guided training` moves to supported as well, and the corresponding fallback wording in the evidence matrix can be retired. If gates 1 or 2 fail, those claims remain blocked and this paper's allowed claims stay exactly where they are today: a reproducible proof-of-concept pipeline with a passing reduced evidence bundle. No claim in this paper depends on the final run passing; the run only decides whether the stronger claims are added. Conversely, the final run is not a shortcut: even a fully passing run does not establish structural load-path viability, superiority over mature baselines, or externally validated aerodynamic optimality, which require the separate structural, baseline, and PDE-validation studies listed in Sections II-B and V.

V. VALIDATION AND TESTING STANDARDS

To keep validation claims bounded, we separate three levels of evidence:

- **Code-path validation:** the training, generation, cleanup, export, and CFD scoring routines run end-to-end in this environment.
- **Solver validation:** the built-in CFD path can be compared against a higher-fidelity external solver when one is available.
- **Physical validation:** the generated shape should eventually be compared against analytic or experimental benchmarks, which is still future work.

A. Verification

Verification checks that the implemented steps are executed as intended. In this revision we verified that:

- the shape-plausibility loss is included in the training objective,
- the connected-component cleanup hook removes smaller disconnected voxel components,
- STL export succeeds from the cleaned voxel grid, and
- the OpenFOAM export hook writes a runnable case directory skeleton, including `blockMeshDict`, `snappyHexMeshDict`, and surface STL output.

B. Validation

Validation is now runnable on a single local machine with OpenFOAM installed. The built-in CFD path is exercised on the reduced CPU pipeline, and the OpenFOAM sonicFoam benchmark is run on the shared centered-cube validation object. The comparison uses the lower-level `forces` output when available, with a manual pressure integration fallback only if the function-object output is missing.

The paper therefore claims the following and nothing more: the export path is concrete, the benchmark comparison is measurable, and the shared validation geometry is documented explicitly so the solver-to-solver check can be repeated without ambiguity.

VI. CONCLUSION

The present implementation demonstrates a working proof-of-concept generative-design pipeline, a reduced freeform-object experiment, and a public VSP Airshow smoke-training run that can be executed in the installed environment. The Airshow addition moves the paper beyond synthetic-only training evidence by converting 355 traceable public geometry records into manifest-backed voxel corpora at 16^3, 32^3, 64^3, and 96^3. It also exposes the current generator's limit: the generated Airshow-checkpoint flight-path samples are nonempty and solver-runnable at the completed resolutions, but they still fail aircraft-specific validity screens such as `span_sanity`. Filtering the 32^3 corpus to 176 source-valid records and fixing a zero-learning-rate resume issue improved the generated samples to a span-sanity-only failure, but it did not produce a passing aircraft-validity result. The 64^3 corpus validates, but the dense decoder did not produce a checkpoint within the local run ceiling. A 96^3 coordinate-decoder run trained for three epochs over the full public corpus and produced one heuristic-screen-passing calibrated top-k export out of three

flight-path cases; this is a useful implementation result, not a reliable aircraft-generation or aerodynamic-validation result.

The direct solver-in-loop follow-up changes what the optimizer actually sees. Earlier runs could print aerodynamic and connectivity numbers without letting those numbers alter the weights. In the new path, scheduled batches decode a full grid, call the internal D3Q27 LBM evaluator, add the measured score to `optimization_loss`, and use two-sided SPSA perturbations as the backward estimate. In the local grid sweep, continuing the 96^3 run drove the scheduled direct solver objective below the 32^3 and 64^3 values, but this remains internal-LBM smoke evidence rather than external PDE validation. The OpenFOAM cross-check completes on the shared centered-cube validation object and gives the repository an external CFD comparison path. The reduced protocol also assembles a passing package-level evidence report with manifest validation, heuristic aircraft-shape checks, grounded condition-response sweeps, manufacturing-bounds checks, and finite baseline statistics. The next revision should therefore prioritize generated-sample validity, solver calibration, PDE comparison, and larger ablation studies before presenting stronger aerodynamic conclusions.

Most importantly, the repository should not yet be described as a fully AI-driven airplane generator conditioned on flight profile, manufacturing method, or structural requirements. The present system is narrower: structured-conditioning plumbing, a reduced evidence package, a grounded public-corpus smoke run, a mixed higher-resolution result, a direct internal-solver training term with black-box SPSA gradients, and a sequential candidate optimizer that can reuse the same measured scores after generation. Stronger claims need stronger evidence: aircraft-like training runs that pass validity gates, conditional ablations, stricter structural and aerodynamic checks, decoder and export-calibration studies, repeated solver-in-loop benchmarks, OpenFOAM or comparable PDE validation, and repeated runs beyond the present reduced local protocol.

The paper therefore closes with a single decisive experiment rather than an open-ended to-do list. The checked-in final protocol (`gcp_128_295m.yaml`) is the one run whose outcome decides whether the three aircraft-level claims – `Generates aircraft structures`, `Conditioned on mission/manufacturing`, and `CFD-guided training` – are added to this paper or remain explicitly future work (Table VIII). The run's success criteria are pre-declared and fully executable with repository tooling, and no claim in this paper depends on the run passing. Whatever its outcome, the repository remains a reproducible proof-of-concept pipeline with a passing reduced evidence bundle; the final run either upgrades those claims on evidence, or leaves the current honest scope exactly as stated.

REFERENCES

[1] A. Elham and M. J. L. van Tooren, "Discrete adjoint aerodynamic shape optimization using symbolic analysis with openfemflow," *Structural and Multidisciplinary Optimization*, vol. 63, pp. 2531–2551, 2021.

[2] E. Andrés-Pérez and C. Paulete-Periáñez, "On the application of surrogate regression models for aerodynamic coefficient prediction," *Complex & Intelligent Systems*, vol. 7, pp. 1991–2021, 2021.

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.

[4] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139. PMLR, 2021, pp. 8162–8171.

[5] A. Nichol, H. Jun, P. Dhariwal, P. Mishkin, and M. Chen, "Point-e: A system for generating 3d point clouds from complex prompts," *arXiv preprint arXiv:2212.08751*, 2022.

[6] H. Jun and A. Nichol, "Shap-e: Generating conditional 3d implicit functions," *arXiv preprint arXiv:2305.02463*, 2023.

[7] J. Wu, O. Sigmund, and J. P. Groen, "Topology optimization of multi-scale structures: a review," *Structural and Multidisciplinary Optimization*, vol. 63, pp. 1455–1480, 2021.

[8] O. Sigmund, "A 99 line topology optimization code written in matlab," *Structural and Multidisciplinary Optimization*, vol. 21, pp. 120–127, 2001.

[9] M. P. Bendsøe and O. Sigmund, *Topology Optimization: Theory, Methods, and Applications*. Springer Science & Business Media, 2013.

[10] I. Sosnovik and I. Oseledets, "Neural networks for topology optimization," *Structural and Multidisciplinary Optimization*, vol. 60, no. 5, pp. 1387–1401, 2019.

[11] A. Jameson, "Aerodynamic design via control theory," *Journal of Scientific Computing*, vol. 3, pp. 233–260, 1988.

[12] ——, "Optimum aerodynamic design via boundary control," in *12th Computational Fluid Dynamics Conference*, 1995, p. 2432.

[13] A. Forrester, A. Sobester, and A. Keane, *Engineering design via surrogate modelling: a practical guide*. John Wiley & Sons, 2008.

[14] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[15] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," *arXiv preprint arXiv:2010.08895*, 2020.

[16] L. Lu, P. Jin, G. Pang, and G. E. Karniadakis, "Learning nonlinear operators via deeponet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, 2021.

[17] Y. Li, Y. Sun, P. Ma, E. Sifakis, T. Du, B. Zhu, and W. Matusik, "Neuralfluid: Neural fluidic system design and control with differentiable simulation," *arXiv preprint arXiv:2405.14903*, 2024.

[18] R. Vinuesa and S. L. Brunton, "Enhancing computational fluid dynamics with machine learning," *Nature Computational Science*, vol. 2, no. 6, pp. 358–366, 2022.

[19] Z. Wang, K. Zhang, J. Chu, and H. Ma, "Physics-informed machine learning for aero-structural design optimization," *Progress in Aerospace Sciences*, vol. 136, p. 100868, 2023.

[20] S. Oh, Y. Jung, S. Kim, I. Lee, and N. Kang, "Deep learning-based generative design for skeleton-optimized structures," *Engineering with Computers*, vol. 35, pp. 1279–1293, 2019.

[21] F. Steinmetz, S. J. Pfertner, and N. R. Gauger, "Generative design of manufacturable structures using differentiable projection," *Computer-Aided Design*, vol. 145, p. 103168, 2022.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[23] J. Wu, C. Zhang, T. Xue, W. T. Freeman, and J. B. Tenenbaum, "Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling," in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[24] S. Luo and W. Hu, "Diffusion probabilistic models for 3d point cloud generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 2837–2845.

[25] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, "Occupancy networks: Learning 3d reconstruction in function space," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4460–4470.

[26] OpenVSP Project, "Openvsp: A parametric aircraft geometry tool," https://github.com/OpenVSP/OpenVSP, 2026, accessed 2026-06-20.

[27] ——, "Openvsp nasa open source agreement license page," https://openvsp.org/license.shtml, 2026, accessed 2026-06-20.

[28] ——, "Vsp airshow," https://airshow.openvsp.org/, 2026, accessed 2026-06-20.

[29] M. Schäfer, S. Turek, F. Durst, E. Krause, and E. Rank, "Benchmark computations of laminar flow around a cylinder," in *Flow Simulation with High-Performance Computers II*. Springer, 1996, pp. 547–566.

[30] M. Geier, A. Pasquali, and M. Schonherr, "The cascaded lattice boltzmann method," *Journal of Computational Physics*, vol. 300, pp. 516–542, 2015.